Week 2 Implementation Plan

OpenClaw + Stratus: Counterfactual Guardrail for Concert Ticket Drops

Track

Stratus X1 Hackathon
Track: `Multi-Agent Systems`

0. Objective

Week 2 should deliver a **90% final product**, not another prototype.

By the end of this week, the demo should prove:

1. **Real incident**
   - a concert ticket drop incident happens in a live environment
   - the issue is observed through real dashboards, alerts, and browser-visible state
2. **Counterfactual guardrail**
   - a naive baseline picks the obvious but dangerous action
   - Stratus chooses a safer action after reasoning over a shortlist
3. **True multi-agent workflow**
   - observation is parallelized across four specialist agents
   - OpenClaw executes the chosen action
   - post-action evaluation writes a new case back into the case library

Product claim:

> We are building a Counterfactual Remediation Guardrail for high-stakes digital drops: a system that predicts whether a remediation is safe before acting, then verifies whether the prediction held after action.

This should feel like a **world-model safety layer for operational agents**, not a generic incident copilot.

1. Week 1 Base We Already Have

We are not starting from zero. The current base already includes:

- an end-to-end OpenClaw + Stratus flow on `workflow`
- live Stratus ranking at the guarded decision step
- local browser execution and Prometheus-backed verification

- plan, browser playbook, and report artifacts
- a retail action library plus scenario shortlist design
- a baseline direction and simulator/comparison shape

Week 2 is about turning that backbone into a polished final product story.

2. Scenario A: Concert Ticket Drop Death Spiral

It is 10:00 AM and a major concert ticket drop opens.

Traffic surges instantly. Users move through queue, seat hold, checkout, and payment. The `payment` path is not fully down, but it becomes slow and flaky under pressure. The system is designed to be resilient, so `checkout` retries aggressively. Users also manually resubmit payment. The result is a retry storm across the critical path.

What the operator sees:

- payment latency rises sharply
- checkout error rate climbs
- queue abandonment begins to increase
- retry factor jumps above baseline
- seats are being held but not converted efficiently
- dashboards turn red even though services still look “up”

What makes the scenario dangerous:

- the obvious human reflex is to restart payment
- that reflex is locally sensible
- but under retry amplification it can make recovery worse
- in ticketing, the system must protect **fairness** and **seat availability**, not just uptime

So the real question is:

> Which action is safe to take first, given the whole system state and the business need to preserve fair access?

What Winning Looks Like

The demo should clearly show:

- the baseline mode recommends `restart_payment`
- the guardrail flow rejects that reflex
- Stratus picks a safer action such as `rate_limit_retries`
- OpenClaw executes that action in the UI
- retry pressure and latency improve
- the final verdict explains what dangerous action was avoided
- the case library stores the full situation, action, and result for future reuse

3. Core Design Choices

Live Demo Base

- **OTel Demo on Kubernetes**
  - gives us a realistic microservice system and browser-visible surfaces
- **Feature-flag trigger path**
  - use the OTel Demo feature UI to create the incident
- **Prometheus + Alertmanager**
  - provide the live alerting and verification loop
- **OpenClaw browser**
  - performs the visible execution step
- **Stratus X1**
  - ranks the shortlisted actions and predicts consequences before action

Frontend Choice

We will not do a full frontend rewrite.

Week 2 frontend choice:

- extend the current FastAPI-based visual surface into one polished **Guardrail Console**
- use current APIs, generated plan/report artifacts, and browser-visible state as the data backbone
- optimize for demo clarity and reliability over framework complexity

Case Library Choice

The product needs a memory layer.

Week 2 case library choice:

- start with a structured local store, not a heavy external system
- recommended v1:
  - `SQLite` or structured `JSONL`
- each record should store:
  - incident fingerprint
  - shortlist
  - chosen action
  - Stratus prediction
  - actual result
  - verdict notes
  - tags for retrieval

This is enough for retrieval, comparison, and demo replay.

4. Multi-Agent Workflow

4.1 Observation Layer: Parallel Sentinel Agents

Observation is the main parallelization moment in the system.

Run these four agents in parallel:

- **Payment Sentinel Agent**
  - reads payment latency, error, timeout, and dependency health signals
- **Queue & Fairness Sentinel Agent**
  - reads queue abandonment, retry factor, fairness skew, and conversion stress
- **Inventory Sentinel Agent**
  - reads seat hold utilization, hold expiration, and inventory lock pressure
- **Browser Sentinel Agent**
  - opens the dashboard and feature UI, captures visible state and screenshots

Each sentinel should output:

- normalized evidence for its category
- key anomaly tags
- retrieved similar cases from the case library
- one short operator-facing summary

4.2 Incident Fingerprint

After the four sentinel agents finish, merge their outputs into one **incident fingerprint**.

This fingerprint should include:

- service and dependency tags
- business-state tags
- anomaly tags
- hard constraints:
  - preserve fairness
  - avoid oversell / seat-lock collapse
  - avoid high blast-radius actions as a first move

This fingerprint is the shared input to planning and evaluation.

4.3 Shortlist Mechanism

The shortlist step is a core Week 2 design problem.

We want:

- action library size: `12–20`
- decision shortlist size: `4–6`

Recommended Shortlist Design: Hybrid

Use a hybrid mechanism, not pure prompting.

1. **Tagged Action Library**
   - each action has metadata:
     - failure modes
     - affected services
     - business impact
     - blast-radius class
     - execution surface
2. **Deterministic Filter**
   - remove actions that do not match the current incident services or failure tags
   - remove actions that cannot be executed in the current environment
3. **Case Retrieval Boost**
   - use case-library similarity to raise actions that worked in similar incidents
   - use case-library warnings to keep dangerous local reflexes in view when needed
4. **Diversity Rule**
   - shortlist must include a spread of action types:
     - one stabilizer
     - one tempting reflex
     - one traffic/degradation option
     - one backup / kill-switch option if relevant
5. **Shortlist Builder Agent**
   - converts the filtered set into the final `4–6` actions
   - writes a rationale explaining why each action made the shortlist

This design keeps the shortlist explainable and stable, while still looking intelligent.

4.4 Guardrail Decision

The **Guardrail Decision Agent** takes:

- incident fingerprint
- `4–6` shortlisted actions
- relevant prior cases
- hard constraints

Then it uses Stratus to produce:

- ranked actions
- confidence
- predicted blast radius
- predicted latency / retry impact
- rollback trigger or stop condition
- final recommended action

4.5 Baseline Mode

Baseline is no longer a permanent agent in the main product story.

Instead, Week 2 should support a **baseline demo toggle**:

- same incident
- same shortlist
- replace Stratus ranking with a naive baseline chooser

The purpose is to show:

- baseline picks the obvious reflex
- guardrail path picks the safer action

4.6 Execution

The **Operator Agent** uses OpenClaw browser to:

- open the console and source dashboards
- inspect the chosen action
- execute the remediation
- refresh and capture post-action evidence

4.7 Evaluation And Learning

The **Evaluation Agent** measures:

- system metrics after action
- browser-visible state after action
- business-relevant signals after action
- predicted vs actual drift

Then it writes a new case into the case library with:

- situation
- shortlisted actions
- chosen action
- predicted result
- actual result
- narrative verdict

This is critical: we are saving full outcomes, not only success/failure.

5. Optional Extension: Abnormality Localization

If we have time, we should extend the system beyond overall guardrailing and let it pinpoint specific abnormalities such as bot attacks.

Recommended Design

Add one post-observation component:

- **Abnormality Attribution Agent**

It consumes the four sentinel outputs and classifies the dominant abnormality, for example:

- payment degradation
- retry storm
- queue fairness distortion

- seat hold clogging
- suspected bot pressure

Suggested Mechanism

Use a hybrid mechanism again:

- rules for obvious signals
  - spike in retry factor
  - fairness skew jump
  - large homepage flood with weak payment conversion
  - abnormal seat hold churn
- agent synthesis for mixed cases
  - when multiple abnormalities overlap

Why It Matters

This gives us a cleaner story than “metrics are bad.”

It lets us say:

> The system did not only choose a safer action. It identified what kind of abnormality was happening and responded accordingly.

6. Frontend Product Surface

Week 2 needs a real frontend, not just dashboards and JSON artifacts.

The target frontend is one operator-facing surface:

> **Guardrail Console**

Required Views

1. **Incident View**
   - incident summary
   - payment / queue / inventory / browser evidence
   - active alerts
   - recent similar cases
2. **Decision View**
   - full action library summary
   - chosen shortlist
   - baseline result vs guardrail result
   - Stratus rationale and predicted blast radius
3. **Execution View**
   - OpenClaw execution log
   - chosen remediation target
   - before / after browser evidence
4. **Verdict View**
   - predicted vs actual
   - dangerous reflex rejected

    - blast radius avoided
    - case saved to library

Frontend Rule

- one polished console
- no disconnected mini-pages
- one owner per view

7. Responsibilities

Everyone must touch both **OpenClaw** and **Stratus**, but with non-overlapping primary ownership.

Hansen

Primary ownership:

- Operator Agent
- Execution flow
- Guardrail Console shell

Develop:

- own the OpenClaw execution handoff and browser playbook integration
- own the `Execution View`
- own overall console shell and navigation
- connect plan -> execute -> verify -> verdict into one clean product flow

Test:

- rehearse end-to-end browser runs
- test fallback path reliability
- test console flow across all four views

Furnish:

- final demo script
- final OpenClaw prompt
- final shell polish

OpenClaw touchpoint:

- deepest owner of browser execution

Stratus touchpoint:

- renders guardrail output into execution and verdict surfaces

Frontend touchpoint:

- Guardrail Console shell + `Execution View`

Tony

Primary ownership:

- Shortlist Builder Agent
- Guardrail Decision Agent
- Stratus schema / prompt / ranking logic

Develop:

- own the `12–20 -> 4–6` shortlist mechanism
- define action metadata schema for shortlist selection
- implement the Stratus input/output contract
- own the decision rationale, veto reasoning, and rollback criteria

Test:

- compare shortlist stability across repeated runs
- compare Stratus prompt variants
- maintain 实验记录 `TBD`

Furnish:

- final shortlist policy
- final Stratus schema
- final reasoning copy for why the dangerous reflex is rejected

OpenClaw touchpoint:

- ensure decision output is directly usable by the Operator Agent

Stratus touchpoint:

- deepest Stratus integration owner

Frontend touchpoint:

- `Decision View`

Charlie

Primary ownership:

- concert-ticket scenario truth
- sentinel schemas
- abnormality attribution extension

Develop:

- write the canonical concert ticket incident spec
- define the four sentinel outputs and anomaly tags
- define case-library taxonomy and retrieval tags
- define what fairness, seat hold stress, and operator-visible success mean
- design the abnormality-localization extension

Test:

- validate that the live demo still feels like a real ticket-drop incident
- validate that browser-visible evidence matches the intended story
- validate that the anomaly labels make business sense

Furnish:

- scenario spec
- UI labels and annotations for incident evidence
- concise product narrative
- abnormality-extension proposal

OpenClaw touchpoint:

- defines what the Browser Sentinel Agent and Operator Agent should inspect

Stratus touchpoint:

- defines the constraints and truths the guardrail should optimize for

Frontend touchpoint:

- `Incident View`

Eason

Primary ownership:

- Evaluation Agent
- baseline demo toggle
- case-library writeback and replay

Develop:

- implement baseline mode on the same shortlist used by Stratus
- own post-action evaluation and verdict generation
- implement case-library persistence format and replay-friendly outputs
- own comparison between:
  - baseline result
  - guardrail result
  - predicted vs actual

Test:

- confirm baseline picks the naive action in Scenario A
- confirm evaluation catches when predicted and actual diverge
- confirm case-library records are understandable and reusable

Furnish:

- final verdict schema
- baseline-vs-guardrail comparison artifact
- final copy for “what happened after action”

OpenClaw touchpoint:

- owns what is shown after execution and verification

Stratus touchpoint:

- compares baseline and Stratus outcomes on the same shortlist

Frontend touchpoint:

- `Verdict View`

8. Frontend Ownership Split

To avoid overlap, frontend work is split by surface:

- **Hansen**
  - shell
  - navigation
  - `Execution View`
- **Tony**
  - `Decision View`
- **Charlie**
  - `Incident View`
- **Eason**
  - `Verdict View`

If time permits, the team can review style together, but ownership remains by view.

9. Repo And Workstreams

Repo:

- `https://github.com/hanshengzhu0001/stratusxpennai`

Branches:

- `workflow`
  - integration
  - OpenClaw execution flow
  - console shell
- `ranking+rollback`
  - shortlist logic
  - Stratus schema and decision logic
- `baseline`
  - baseline mode
  - evaluation
  - case-library writeback / replay
- `scenario+eval`
  - scenario truth
  - sentinel schema
  - anomaly extension

- UI labels

Recommended base setup:

```
git clone -b workflow
        https://github.com/hanshengzhu0001/stratusxpennai.git
cd stratusxpennai
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
git fetch origin
```

Independent starting points:

- Tony can start immediately on shortlist and Stratus logic
- Charlie can start immediately on scenario, sentinel schema, and anomaly design
- Eason can start immediately on baseline mode and verdict/case-library schema
- Hansen can start immediately on OpenClaw execution flow and console shell

10. Demo Prep: How We Differentiate And Impress

This section matters as much as the code.

What Makes The Project Different

We are not just "using Stratus to pick an action."

We are showing:

- a dangerous local reflex is rejected
- the product protects fairness and seat access, not just uptime
- OpenClaw visibly acts in the browser
- the system learns by saving the full case after action

Demo Structure

The cleanest live demo should be:

1. Trigger the concert ticket incident live
2. Show all four sentinel panels update
3. Show similar prior cases retrieved from the case library
4. Turn on baseline mode and show it picks `restart_payment`
5. Switch to guardrail mode and show Stratus picks the safer action
6. Let OpenClaw execute the action
7. Show the verdict and save the case

What Judges Should Remember

- "The obvious fix was wrong."
- "The system knew why it was wrong before acting."
- "The agent acted in the UI and then verified the result."

- “It protects fairness, not just latency.”

Rehearsal Checklist

- one polished Guardrail Console
- one clean baseline-vs-guardrail toggle
- one visible OpenClaw execution moment
- one before / after verdict slide inside the app
- one backup local-control-plane run recorded in case the K8s demo is flaky

11. Grounding / Sources

Official references supporting the live demo path:

- OpenTelemetry Demo scenarios and feature-flag path
  - https://opentelemetry.io/docs/demo/scenarios/
- OpenTelemetry Demo repository
  - https://github.com/open-telemetry/opentelemetry-demo
- Prometheus Alertmanager documentation
  - https://prometheus.io/docs/alerting/latest/alertmanager/
- Kubernetes Horizontal Pod Autoscaler documentation
  - https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- OpenFeature overview
  - https://openfeature.dev/docs/reference/intro/